Parallel Sorting Algorithms On Hybrid Heterogeneous Servers

Contents

Abstract **3**

1 Introduction **3**

2 Related Work **3**

3 Architecture Specifications **4**
- 3.1 Heterogeneous Systems . . . 4
- 3.2 Nvidia GPUs . . . 4
- 3.3 Intel CPUs . . . 4

4 Tools For Parallel Programming **4**
- 4.1 OpenMP . . . 4
- 4.2 OpenACC . . . 4
- 4.3 CUDA . . . 5
- 4.4 NVIDIA Thrust . . . 5
- 4.5 Pthreads . . . 5
- 4.6 Intel TBB . . . 5
- 4.7 Intel OneAPI . . . 5
- 4.8 Compilers . . . 5
  - 4.8.1 NVCC . . . 6
  - 4.8.2 ICPX . . . 6

5 Parallel Sorting Algorithms **6**
- 5.1 Definition Of Sorting . . . 6
- 5.2 Types Of Sorting Algorithms . . . 6
- 5.3 Time Complexity . . . 7
- 5.4 Testing On Sequential CPU . . . 7
- 5.5 Testing On Parallel CPU . . . 7
- 5.6 Testing On Parallel GPU . . . 8

6 Analysis of Parallel Radix Sort Algorithm **8**

7 Performance On A Heterogeneous System **9**
- 7.1 Data Partitioning . . . 9

7.2 Measuring Million Elements Per Second with Input Array Size 10
7.3 Applying Algorithm 11
7.3.1 Phase 1 12
7.3.2 Phase 2, 3 4 12
7.3.3 Phase 5 13
7.4 Measuring Million Elements Per Second with Input Integer Length 13

8 Application Of the Partitioning Algorithm 15

9 Conclusion 16

Abstract

Sorting is one of the fundamental algorithms in computer science used in almost any area of computing with a large amount of data points or elements. The importance of efficient sorting algorithms are becoming increasingly important as the size and scale of data continues to grow.

Hybrid heterogeneous servers featuring multicore CPUs and multiple accelerators such as GPUs dominate the computing landscape due to their ability to efficiently handle very large inputs by performing multiple operations at once.

Because of their importance, efficient sorting algorithms have been the subject of much computer science research and many of the large CPU and GPU manufacturers such as Intel and NVIDIA have released mature and highly optimized sorting libraries such as NVIDIA's Thrust and Intel OneAPI. The goal of this paper is to make efficient use of these libraries on a hybrid heterogeneous system.

1 Introduction

As datasets continue to grow in scale, with the rise of large datasets and increase in data storage capabilities, the performance of sorting algorithms become increasingly important, and as the hardware landscape evolves our implementations become increasingly complicated. Modern high performance servers are no longer homogeneous - they are hybrid systems featuring multicore CPUs alongside one or more GPU accelerators, each with distinct memory hierarchies, parallelism models and performance characteristics.

Making full use of such a system is a non-trivial issue. A naive approach might run sorting entirely on the GPU, leaving the CPU idle, or diviide the input array equally between processors regardless of their different processing speeds, both of these strategies will cause some of our processors to sit idly and leave performance on the table. The fundamental challenge is given an input array of n elements and a set of p processors, how do we partition the work so that all processors finish at the same time?

2 Related Work

- *Programming Massively Parallel Processors, Version 4* — David B. Kirk, Wen-mei W. Hwu
- *High Performance Heterogeneous Computing* — Alexey L. Lastovetsky and Jack J. Dongarra
- OpenH: A Novel Programming Model and API for Developing Portable Parallel Programs on Heterogeneous Hybrid Servers [2]

3 Architecture Specifications

3.1 Heterogeneous Systems

A heterogeneous server is a server featuring multicore CPU processors hosting multicore accelerators. A hybrid parallel program signifies a program that divides the workload heterogeneously between the multicore CPU processors and accelerators of a server to provide maximum resource utilization and performance on the server.

3.2 Nvidia GPUs

Our system contains two NVIDIA A40 GPUs. NVIDIA A40 GPUs are equiped with 48 GiB of GDDR6 memory access, a 384-bit bus, delivering up to 696 GiB/s peak memory bandwidth. Via NVLink two A40s can be bridged together to expand usable memory to 96 GiB, making it well suited for large datasets anc complex models. [1]

3.3 Intel CPUs

Our system contains one Intel Icelake multicore CPU - their processors are manufactured on Intel's 10nm process and feature two AVX-512 units per core, enabling up to 16 double precision FLOPS per cycle.

4 Tools For Parallel Programming

4.1 OpenMP

OpenMP is a parallel programming model for shared memory and distributed shared memory multiprocessors. It is comprised of a set of compiler directives that describe the parallelisms in the source code, along with a supporting library of subroutines available to the application.

The directives are instructional notes to any compiler supporting OpenMP. They take the form of source code comments (in Fortran) or `#pragma` (in C/C++) in order to enhance application portability when porting to non-OpenMP environments. [3]

4.2 OpenACC

In contrast to current mainstream GPU programming, such as CUDA and OpenCL, where more explicit compute and data management is necessary, porting of legacy CPU-based applications with OpenACC requires only code annotations without any significant structural changes in the original code, which allows considerable simplification and productivity improvement when hybridizing existing applications.

Programming with OpenACC directives, while greatly simplified, is not as flexible as using CUDA or OpenCL. For example, both CUDA and OpenCL provide fine-grained synchronization primitives, such as thread synchronization and atomic operations, whereas OpenACC does not. [5]

4.3 CUDA

CUDA is a minimal extension of the C and C++ programming languages. A programmer writes a serial program that calls parallel kernels, which can be simple functions or full programs. The CUDA program executes parallel kernels across a set of parallel threads on the GPU. The programmer organizes these threads into a hierarchy of thread blocks and grids.

The CUDA programming model is similar in style to a single-program multiple data (SPMD) software model — it expresses parallelism explicitly, and each kernel executes on a fixed number of threads. However, CUDA is more flexible than most SPMD implementations because each kernel call dynamically creates a new grid with the right number of thread blocks and threads for the application step.

4.4 NVIDIA Thrust

Thrust is a C++ template library for CUDA based on the Standard Template Library (STL). Thrust allows you to implement high performance parallel applications with minimal programming effort through a high-level interface that is fully interoperable with CUDA C. Thrust includes support for algorithms such as sort, scan, transform and reduce, and these algorithms have been highly optimized and run more efficiently than a single developer could produce on their own in a reasonable amount of time.

4.5 Pthreads

Pthreads is a standardized model for dividing a program into substacks whose execution can be interleaved or run in parallel. The "P" in Pthreads come from POSIX (Portable Operating System Interface), the family of IEEE operating system interface standards in which Pthreads is defined. [4]

4.6 Intel TBB

Intel Threading Building Blocks is a C++ runtime library that abstracts the low-level threading details necessary for effectively utilizing multi-core processors. It uses C++ templates to eliminate the need to create and manage threads. The library consists of data structures and algorithms that simplify parallel programming in C++ by avoiding requiring a programmer to use native threading packages such as POSIX. [8]

4.7 Intel OneAPI

4.8 Compilers

Due to many of the programming models for efficient parallel programming on CPUs and GPUs being vendor locked and managed by companies for specific use on their hardware, in a heterogeneous system it can be necessary to use multiple compilers, compile files separately and then link them together.

4.8.1 NVCC

NVIDIA CUDA Compiler is a proprietary compiler by NVIDIA intended for use with CUDA. The compilation trajectory involves several splitting, compilation, preprocessing and merging steps. It is the purpose of NVCC to hide the intricate details of CUDA compilation from developers. All non-CUDA compilation steps are forwarded to a C++ host compiler that is supported by NVCC, and NVCC translates its options to appropriate host compiler command line options. [6]

4.8.2 ICPX

Intel C++ Compiler is a proprietary compiler by Intel intended for use across Intel processor architectures. The compilation process leverages advanced optimization techniques including auto-vectorization, interprocedural optimization, and profile-guided optimization steps. It is the purpose of the Intel C++ Compiler to maximize application performance on Intel hardware by exposing low-level architectural features to developers. Full compatibility with standard C++ host toolchains is maintained, and the compiler translates its optimization flags to appropriate platform-specific instructions targeting Intel's SSE, AVX and other SIMD instruction sets.

5 Parallel Sorting Algorithms

5.1 Definition Of Sorting

Sorting is one of the earliest applications for computers. A sorting algorithm arranges the elements of a list into a certain order. The order to be enforced by a sorting algorithm depends on the nature of these elements. Any sorting algorithm must satisfy the following two conditions:

1. The output is in either nondecreasing or nonincreasing order. For nondecreasing order, each element is no smaller than the previous element according to the desired order. For nonincreasing order, each element is no larger than the previous element according to the desired order.
2. The output is a permutation of the input. That is, the algorithm must retain all of the original input elements while reordering them into the output. [7]

5.2 Types Of Sorting Algorithms

Sorting algorithms can be classified into stable and unstable algorithms. A stable sort algorithm preserves the original order of appearance when two elements have equal key value.

Sorting algorithms can also be classified into comparison-based and non-comparison-based algorithms. Comparison sorting algorithms arrange elements by comparing pairs and cannot achieve a time complexity greater than $O(n \log n)$. Non-comparison algorithms exploit numerical properties to achieve a faster and potentially linear $O(n)$ time.

Table 1: Classification of parallel sorting algorithms

	Comparison Based	**Non-Comparison Based**
Stable	Parallel Merge Sort	LSD Radix Sort
Unstable	Parallel Quick Sort	MSD Radix Sort

5.3 Time Complexity

For sorting of large datasets we want to pick algorithms with linear time complexity. Radix sort contains downsides for sorting datasets with large integer values, but parallel sorting on a heterogeneous system will usually be done on datasets where the size of the input makes time complexity more important.

Table 2: Time complexity of parallel sorting algorithms

Sorting Algorithm	**Best Case**	**Average Case**	**Worst Case**
LSD Radix Sort	$O(n \cdot d)$	$O(n \cdot d)$	$O(n \cdot d)$
MSD Radix Sort	$O(n)$	$O(n \cdot d)$	$O(n \cdot d)$
Parallel Merge Sort	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$
Parallel Quick Sort	$O(n \cdot \log(n))$	$O(n \cdot \log(n))$	$O(n^2)$

5.4 Testing On Sequential CPU

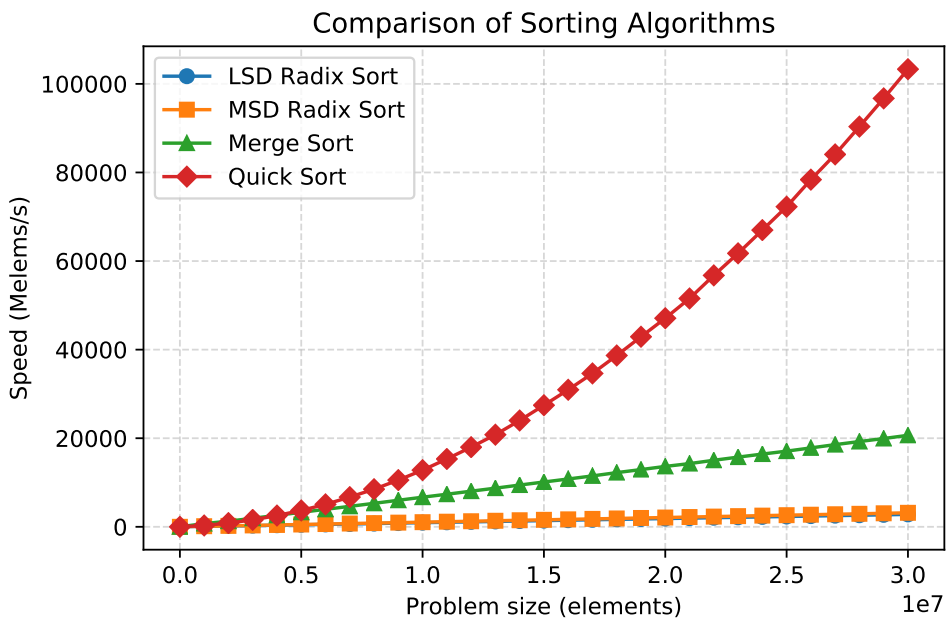

Figure 1: Comparison of sorting algorithms

5.5 Testing On Parallel CPU

Here we can see that radix sort on a parallel CPU maintains the linear growth of squeal radix sort.

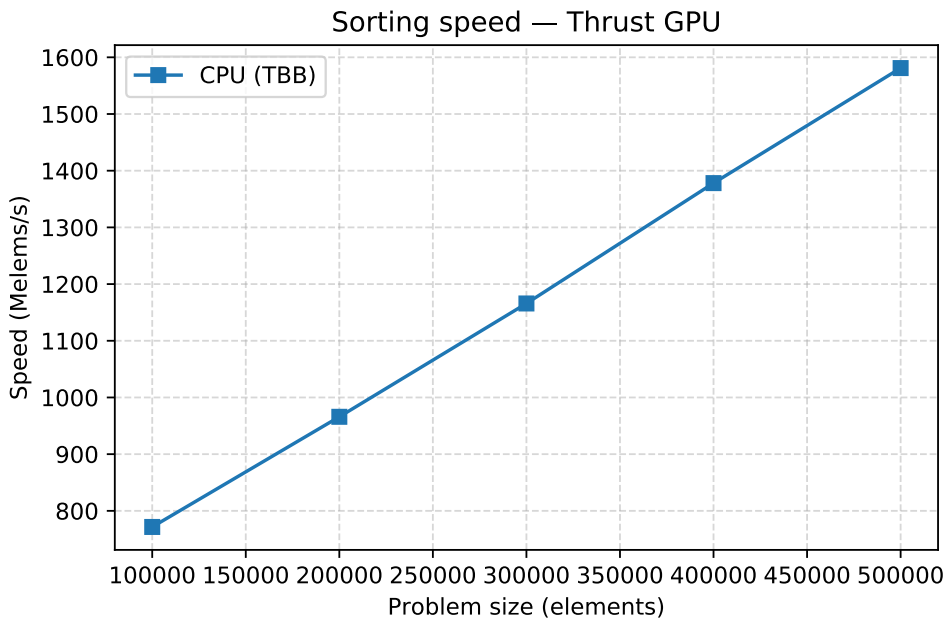

Figure 2: Sorting speed on CPU with Intel TBB

5.6 Testing On Parallel GPU

And on GPUs the growth still stays linear.

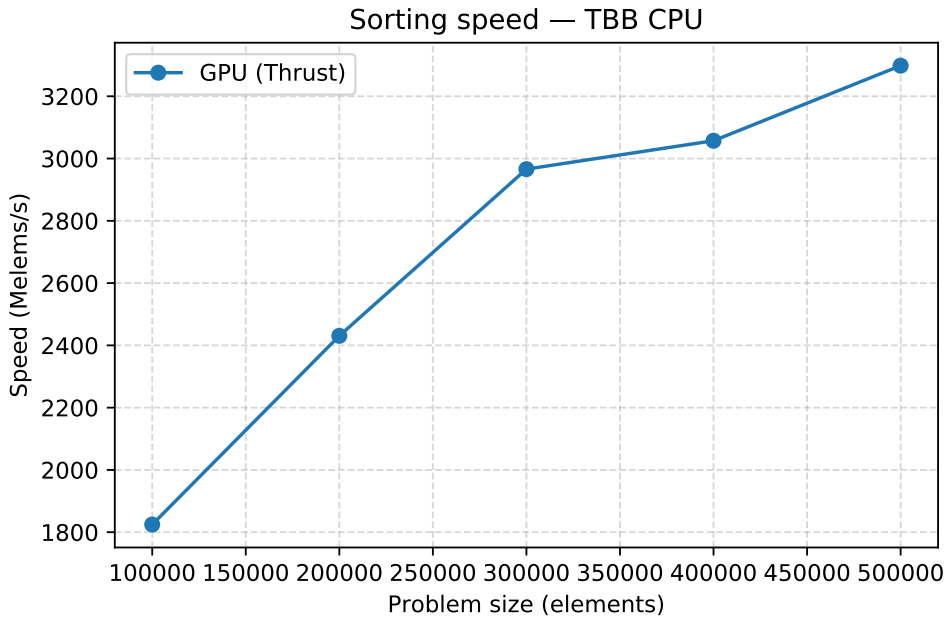

Figure 3: Sorting speed comparison on GPU with NVIDIA Thrust

6 Analysis of Parallel Radix Sort Algorithm

To further understand why these trend lines appear, we can analyze the Radix Sort Implementation in Chapter 13 of Programming Massively Parallel Processors.

```
__global__ void radix sort_iter(unsigned int* input, unsigned int*
    output, unsigned int* bits, unsigned int N, unsigned int iter){
    unsigned int i = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int key, bit;
    if(i < N){
        key = input[i];
        bit = (key >> iter) & 1;
        bits[i] = bit;
    }

    exclusiveScan(bits, N);
    if(i < N){
        unsigned int numOnesBefore = bits[i];
        unsigned int numOnesTotal = bits[i];
        unsigned int dst = (bit == 0) ? (i - numOnesBefore)
                                      : (N - numOnesTotal -
                                         numOnesBefore);
        output[dst] = key;
    }
}
```

7 Performance On A Heterogeneous System

An immediate implication from the heterogeneity of processors in a network of computers is that the processors run at different speeds. A good parallel application for a homogeneous distributed memory multiprocessor system tries to evenly distribute computations over available processors.

A good parallel application for the heterogeneous platform must distribute computations unevenly, taking into account the difference in processor speed. The faster the processor is, the more computations it must perform.

The speed is defined as the number of computation units performed by the processor per one time unit. The model is application specific. In particular, this means that the computation unit can be defined differently for different applications — the important requirement is that the computation unit does not have to vary during the execution of the application.

7.1 Data Partitioning

In High Performance Heterogeneous Computing we are presented with the Functional Performance Model for heterogeneous processors

7.2 Measuring Million Elements Per Second with Input Array Size

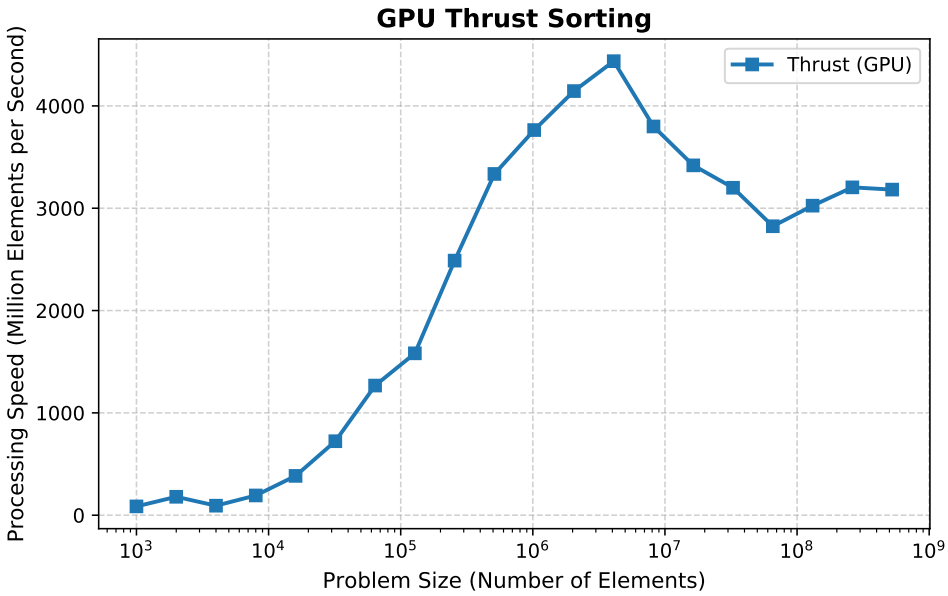

Figure 4: GPU Thrust Sorting Speed in Logarithmic Graph

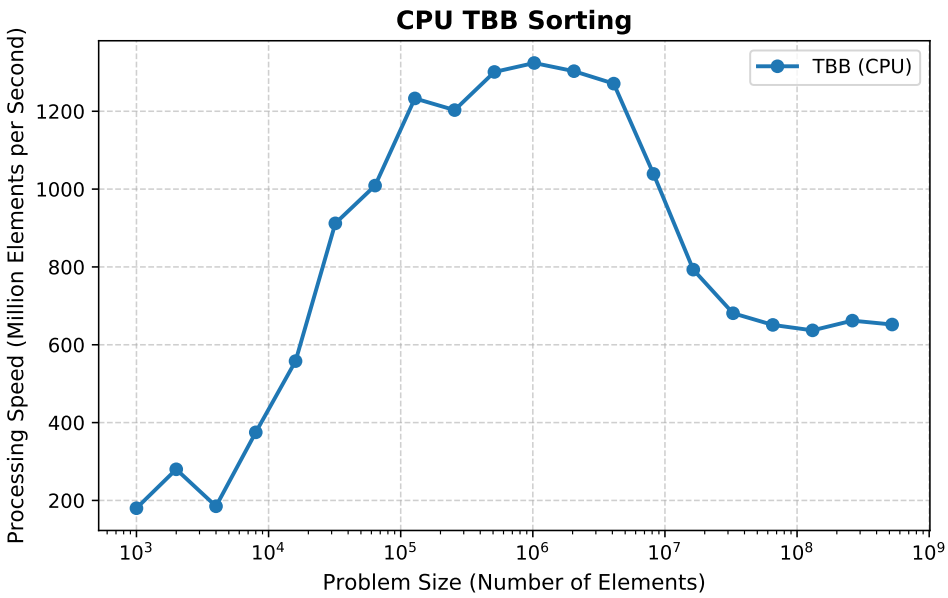

Figure 5: CPU TBB Sorting Speed in Logarithmic Graph

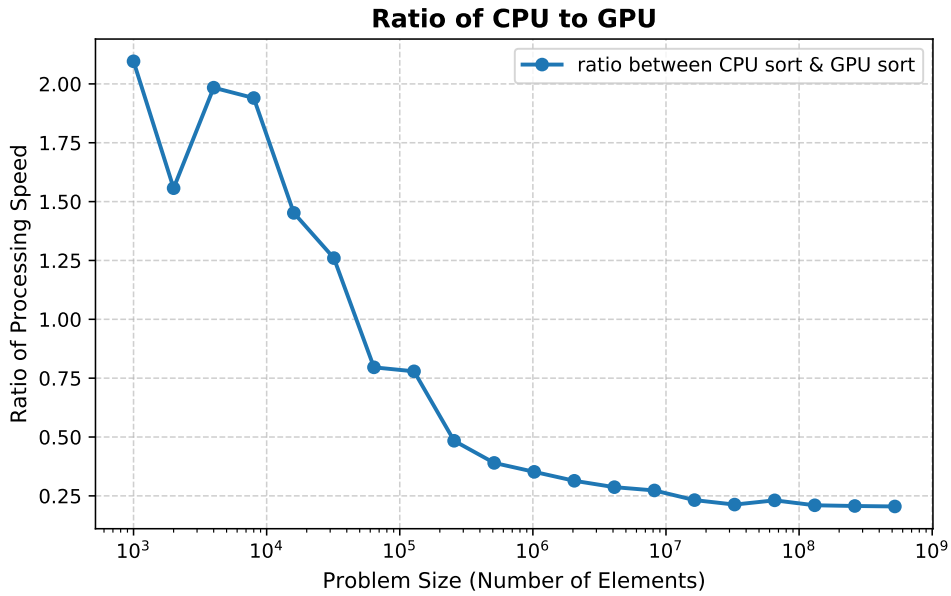

Figure 6: Ratio Between CPU and GPU Sorting Speed

The results shown in Figures 4, 5, and 6 reveal a clear performance relationship between CPU and GPU sorting that directly informs our data partitioning strategy for heterogeneous systems and fits quite well into Lastovetsky and Reddy functional performance model.

Both Thrust and TBB display the shape which the paper assumes: a rise phase, a peak then decline. This matches the paper's assumption that speed is a continuous, unimodal function of problem size, not a constant. A "single number" model would fail badly here.

The Thrust/TBB ratio drops from 2.1 at small sizes all the way to 0.2 at large sizes which is nearly a 10x swing. A traditional model picking one ratio would either badly over-assign or under-assign work to the GPU.

The chunk size should be 1024

7.3 Applying Algorithm

From Latovetsky and Reddy 2007

Algorithm 1.1 Approximation of the n_i so that either $n_i = \lfloor x_i \rfloor$ or $n_i = \lfloor x_i \rfloor - 1$ for $1 \leq i \leq p$ where $\frac{x_1}{s_1(x_1)} = \frac{x_2}{s_2(x_2)} = ... = \frac{x_p}{s_p(x_p)}$ and $x_1 + x_2 + ... + x_p = n$:

1. The upper line U is drawn through the points $(0, 0)$ and $(\frac{n}{p}, max_i\{s_i\{\frac{n}{p}\}\})$, and the lower line L is drawn through the points $(0, 0)$ and $(\frac{n}{p}, min_i\{s_i\{\frac{n}{p}\}\})$, as shown in Figure 7.

2. Let $x_i^{(U)}$ and $x_i^{(L)}$ be the coordinates of the intersection points of lines U and L with the function $s_i(x)(1 \leq i \leq p)$. If there exists $i \in 1, ..., p$ such that $x_i^{(L)} - x_i^{(U)} \geq 1$ then go to step 3 else go to step 5.

3. Bisect the angle between lines U and L by the line M. Calculate coordinates $x_i^{(M)}$ of the intersection points of the line M with the function $s_i(x)$ for $1 \leq i \leq p$.

4. If $\sum_1^p x_i^{(M)}$ then U = M else L = M. Repeat step 2.

5. Approximate the n_i so that $n_i = \lfloor x_i^{(U)} \rfloor$ for $1 \leq i \leq p$.

7.3.1 Phase 1

The vertical blue line correctly marks $n/p = 524,288,000/3 \approx 1.75 * 10^8$.

The green ray (U) passes through the origin and the point (n/p, thrust speed at (n/p)) - the max speed at n/p which is the GPU at $\sim$ 3182.

The red ray (L) passes through the origin and the point (n/p, tbb speed at (n/p)) - the max speed at n/p which is the CPU at $\sim$ 650.

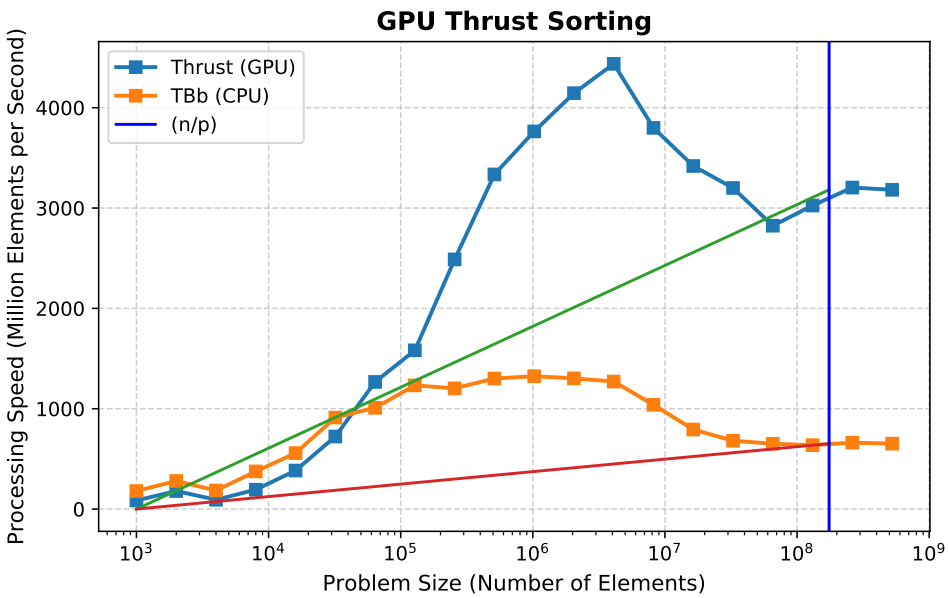

Figure 7:

7.3.2 Phase 2, 3 4

This phase of the algorithm effectively performs a binary search on the two slopes, looking for the unique slope where the intersection sums exactly to n. This algorithm runs in a time complexity of O(log n) - just like binary search, it eliminates half the remaining solution space on every iteration.

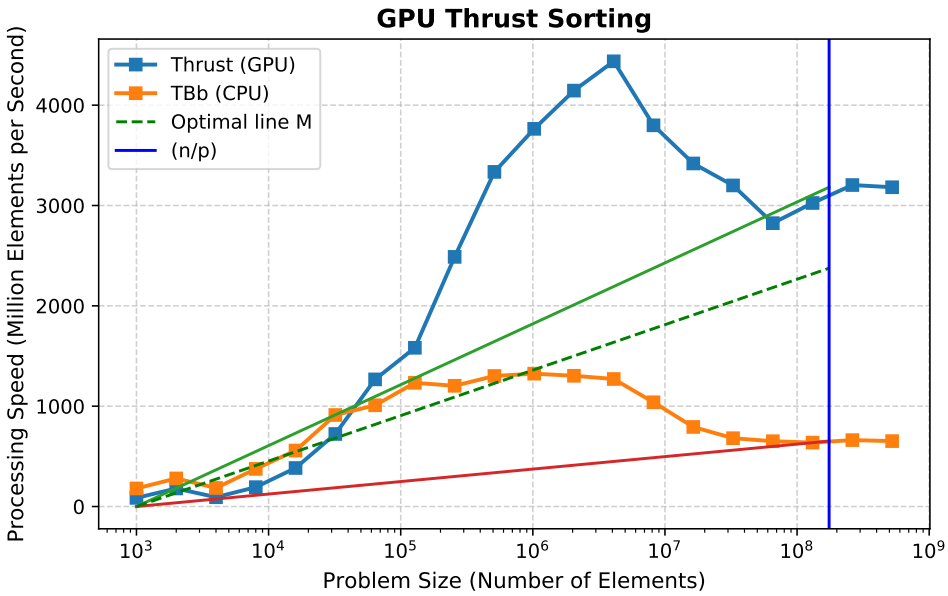

Figure 8:

7.3.3 Phase 5

Algorithm 1.2 Iterative incrementing of some n_i until $n_1 + n_2 + ... + n_p = n$:

1. If $n_1 + n_2 + ... + n_p < n$ then go to step 2 else stop the algorithm
2. Find $k \in \{1, ..., p\}$ such that $\frac{n_k+1}{s_k(n_k+1)} = min_{i=1}^{p} = \{\frac{n_i+1}{s_i(n_i+1)}\}$.
3. $n_k = n_k + 1$ Repeat step 1.

Writing this algorithm in pseudocode gives us

Algorithm 1 Algorithm 1.2

n — total chunks, n_i — initial allocations, processors with speed functions $s_i(x)$ n_i — optimal chunk allocation summing to n

while true **do** totalSum $\leftarrow \sum_{i=1}^{p} n_i$
  if totalSum $\geq n$ **then return** n_i minimum $\leftarrow \infty$ $k \leftarrow 0$
    for $i = 1$ p **do** $t \leftarrow \dfrac{n_i + 1}{s_i(n_i + 1)}$
      if $t <$ minimum **then** minimum $\leftarrow t$ $k \leftarrow i$ $n_k \leftarrow n_k + 1$

7.4 Measuring Million Elements Per Second with Input Integer Length

Because the time complexity of radix sort is O(n*d), unlike most sorting algorithms which have a sorting algorithm of O(n * log(n)) the length of the integers being sorted will have an impact on the MFLOPS. Unlike when observing the trend for the Input Array Size here we see that as the size of the integers being sorted increases the MFLOPS decreases.

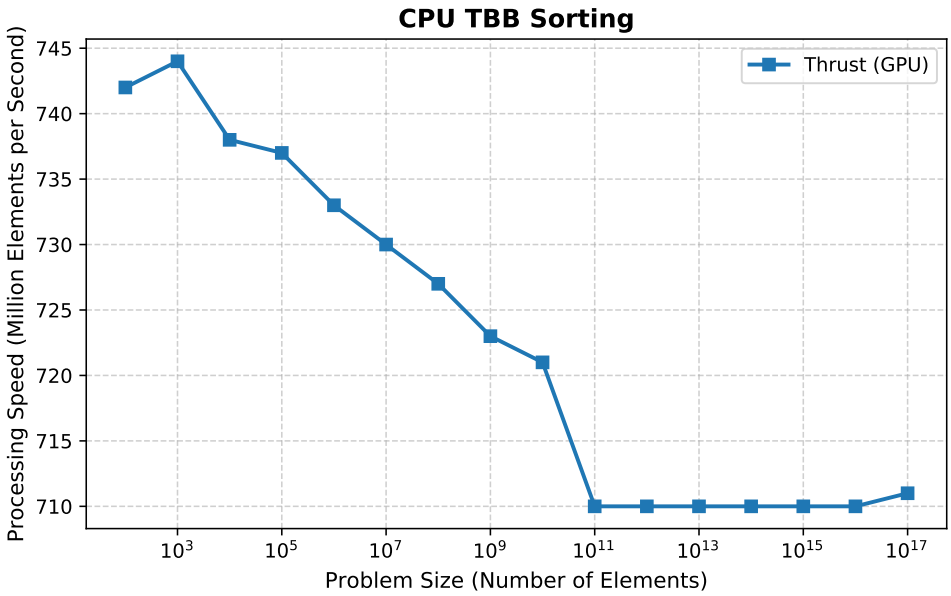

Figure 9:

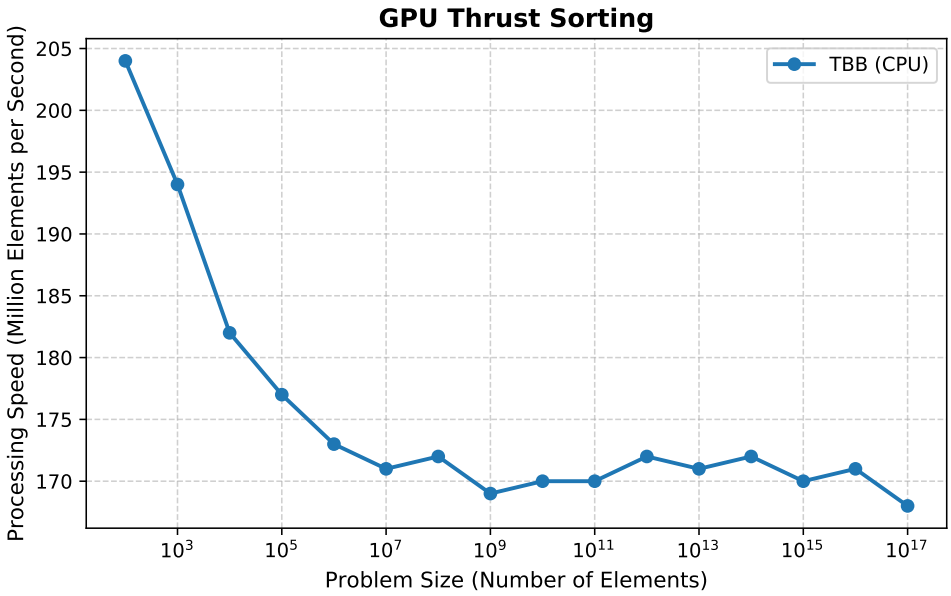

Figure 10:

However, we see a similar pattern where there is a sharp change until it reaches a certain threshold where it begins to flatline.

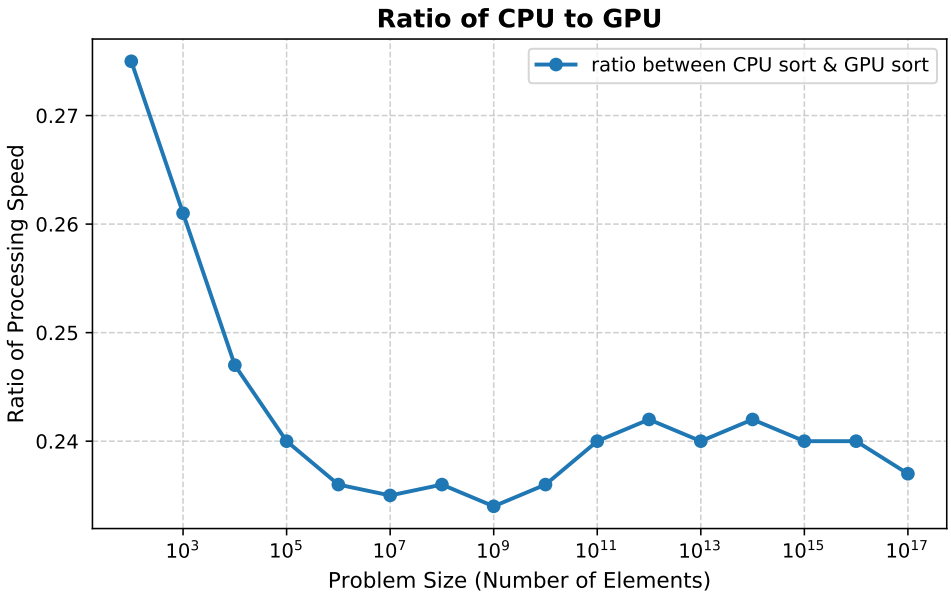

Figure 11:

Here we can see a slight change in the ratio between the CPU Radix sort and the GPU radix sort as the problem size increases. However, this change is very small, only going from 0.27 to 0.24 and becomes a constant ratio after a certain point. From this we can assume that our heterogeneous system is should be able to assume the same ratio regardless of problem size.

8 Application Of the Partitioning Algorithm

In this section, we apply the set partitioning algorithm to a parallel sorting algorithm application using partitioning of an input array on a network of p heterogeneous computers. Our aim in this section is not to show how radix sort algorithms can be efficiently parallelized but to explain in simple terms how the set partitioning algorithm using the functional model can be applied to optimally schedule computational tasks on a network of heterogeneous computers.

Table 3: Runtime comparison across functional model and benchmark runs (ms)

Array Size	**GPU 1 (ms)**	**GPU 2 (ms)**	**CPU (ms)**	**Spread (ms)**	**Balance**
100000000	94	82	81	12	86.3%
200000000	193	200	174	25	87.2%
300000000	294	243	291	50	82.7%
400000000	387	336	329	58	84.9%
500000000	497	411	404	93	81.3%

Table 4: Runtime comparison across in functional model (ms)

Array Size	GPU 1 (ms)	GPU 2 (ms)	CPU (ms)	Spread (ms)	Balance
100000000	80	74	219	144	34.1&
200000000	172	152	454	302	33.6&
300000000	261	243	705	461	34.5%
400000000	351	299	1002	702	29.9%
500000000	459	432	1310	878	33.0%

9 Conclusion

In this report, I looked at the problem of a optimal distribution of sorting algorithms on a network of heterogeneous computers, making use of optimally parallelized sorting algorithms and distributing the workload across processors when their relative speeds vary with the problem size. This involved looking at different sorting algorithms and seeing which is most efficient for sorting large inputs when making us of parallel computing techniques. After implementing these efficient sorting algorithms we need to make use of a functinoal performance model to effectively distribute our workload.

References

[1] NVIDIA Corporation, *NVIDIA A40 GPU Accelerator Product Brief*, PB-09976-001_v08, April 2022.

[2] S. Farrelly, R. R. Manumachu and A. Lastovetsky, *OpenH: A Novel Programming Model and API for Developing Portable Parallel Programs on Heterogeneous Hybrid Servers*, 2024.

[3] Rohit Chandra, *Parallel Programming in OpenMP*, 2000.

[4] Dick Buttlar, Jacqueline Farrell, and Bradford Nichols, *Pthreads Programming: A POSIX Standard for Better Multiprocessing*, O'Reilly Media, Inc., 1996.

[5] T. Hoshino, N. Maruyama, S. Matsuoka and R. Takaki, *CUDA vs OpenACC: Performance Case Studies with Kernel Benchmarks and a Memory-Bound CFD Application.*

[6] NVIDIA Corporation, *CUDA Compiler Driver NVCC.*

[7] Wen-mei W. Hwu, Izzat El Hajj, *Programming Massively Parallel Processors*, Chapter 13.

[8] Chuck Pheatt, *Intel® Threading Building Blocks*, Journal of Computing Sciences in Colleges, 23(4):298–298, 2008.